

项目指导书

(三级)

网络点餐系统

（一）基本信息

基 于 Java Web 开 发 环 境 ， 利 用 HTML，CSS，Servlet，JDBC，JSP2.0 及 JSTL 等技术实现网络点餐系统的部分功能。

1、类型：

■ 综合类 ☐ 设计类 ☐ 创新类 ☐ 验证类

2、学时安排：

课上 16 学时。

3、项目目标（预期学习效果）：与 KTA 点对应描述

（1）能够分析网络点餐系统的项目需求，针对系统的需求，正确选择开发与管理工具，并完成现代开发工具、Web 应用服务器以及数据库服务器的安装及配置；

（2）能够依据系统的设计方案，熟练地应用 Servlet 技术、JDBC 技术、

JSP2.0 技术等实施网络点餐系统的开发。

KTA 点：KTA1 项目导引-网络点餐系统、KTA2Java Web 开发环境配置和使用、KTA3HTML 文档结构、KTA4HTML 常用文本标签、KTA5HTML 多媒体标签、KTA6HTML 表格、KTA7HTML 表单、KTA8HTML5 常用标签、KTA9CSS、KTA11HttpServletRequest、KTA12HttpServletResponse、KTA13HttpSession、KTA20DriverManager 接口、KTA21Connection 接口、KTA23PreparedStatement 接口、KTA24ResultSet 接口、KTA25 使用 JDBC 连接数据库、KTA26JDBC 操作的封装、KTA27 使用 Servlet 和 JDBC 完成实现动态网页、KTA29JSP 指令元素、KTA33EL 表达式语言、KTA34EL 运算符和操作符、KTA35EL 隐含对象、KTA37<c:out> 标签、KTA38<c:set> 标签、KTA39<c:if> 标签、KTA40<c:choose><c:when><c:otherwise> 标签、KTA41<c:forEach> 标签、KTA42MVC 模式、KTA43 基于 MVC 模式实现网络点餐系统。

（二）组织形式

课上实施，每名学生单独完成环境搭建，系统开发，测试及运行。

开发环境：JDK8 及以上，Eclipse4.12 及以上，Tomcat9.0 及以上，Navicat12 及以上，MySQL8.0 及以上。

（三）任务描述

开发一个网络点餐系统，要求实现如下功能：

- （1）用户在未登录的状态下可以浏览点餐系统中的热点菜品（按点餐率升序排列），今日特价菜品和厨师推荐菜品。
- （2）用户可以登录点餐系统，如果是新用户则需要先注册再登录。
- （3）登录后的用户可以按照菜品分类浏览系统中的所有菜品，并可以批量挑选喜爱的菜品加入点餐车。
- （4）登录后的用户可以查看自己的点餐车浏览所点的菜品，并可以批量删除。
- （5）登录后的用户可以修改自己的资料。

- (6) 登录后的用户可以退出点餐系统。
- (7) 系统提供一个管理员用户，管理员登录后可以行使管理功能。
- (8) 登录后的管理员可以对用户进行管理，可以浏览所有用户的信息，并对用户进行删除。
- (9) 登录后的管理员可以对菜品分类进行管理，可以浏览所有菜品分类，并对菜品分类进行增加、删除和修改。
- (10) 登录后的管理员可以对菜品进行管理，可以浏览所有菜品的信息，并对菜品进行增加、删除和修改。
- (11) 登录后的管理员可以查看所有用户的点餐情况。

分模块实现网络点餐系统部分功能的开发，具体任务如下：

- (1) 任务 1：Java Web 开发和运行环境的搭建；
- (2) 任务 2：网络点餐系统的页面原型；
- (3) 任务 3：网络点餐系统的用户注册和登录；
- (4) 任务 4：网络点餐系统的菜品分类管理；
- (5) 任务 5：网络点餐系统的菜品分类管理的优化。

（四）指导内容

1、系统模块设计：

依据任务描述，将网络点餐系统划分为 3 个模块，分别是：公共模块，用户模块和管理员模块，如图 1 所示。

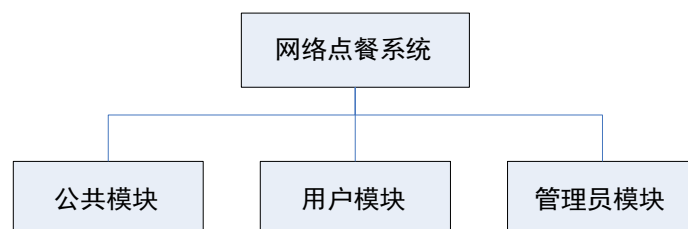


图 1 系统模块结构图

公共模块分为首页、登录、注册、结果页和退出 5 个部分，如图 2 所示。

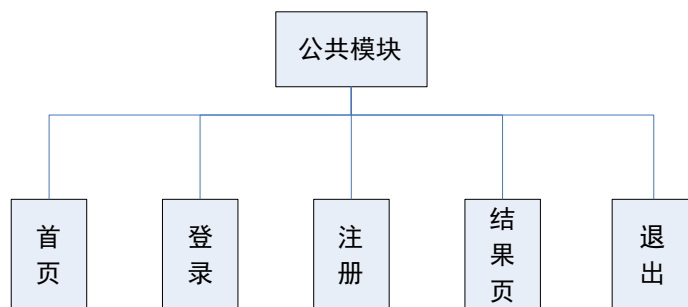


图 2 公共模块细化图

用户模块分为用户首页（即点餐页面）、点餐车页面和修改用户资料 3 个部分，如图 3 所示。

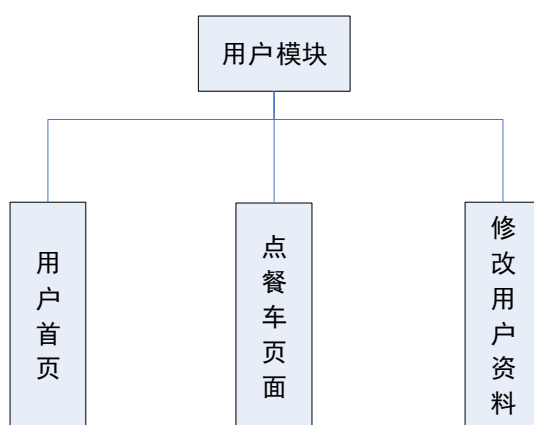


图 3 用户模块细化图

管理员模块分为管理员首页、用户管理、菜品分类管理、菜品管理、查看用户点餐情况 5 个部分，其中用户管理包括用户浏览和用户删除；菜品分类管理包括菜品分类浏览、菜品分类添加、菜品分类修改和菜品分类删除；菜品管理包括菜品浏览、菜品添加、菜品修改和菜品删除，如图 4 所示。

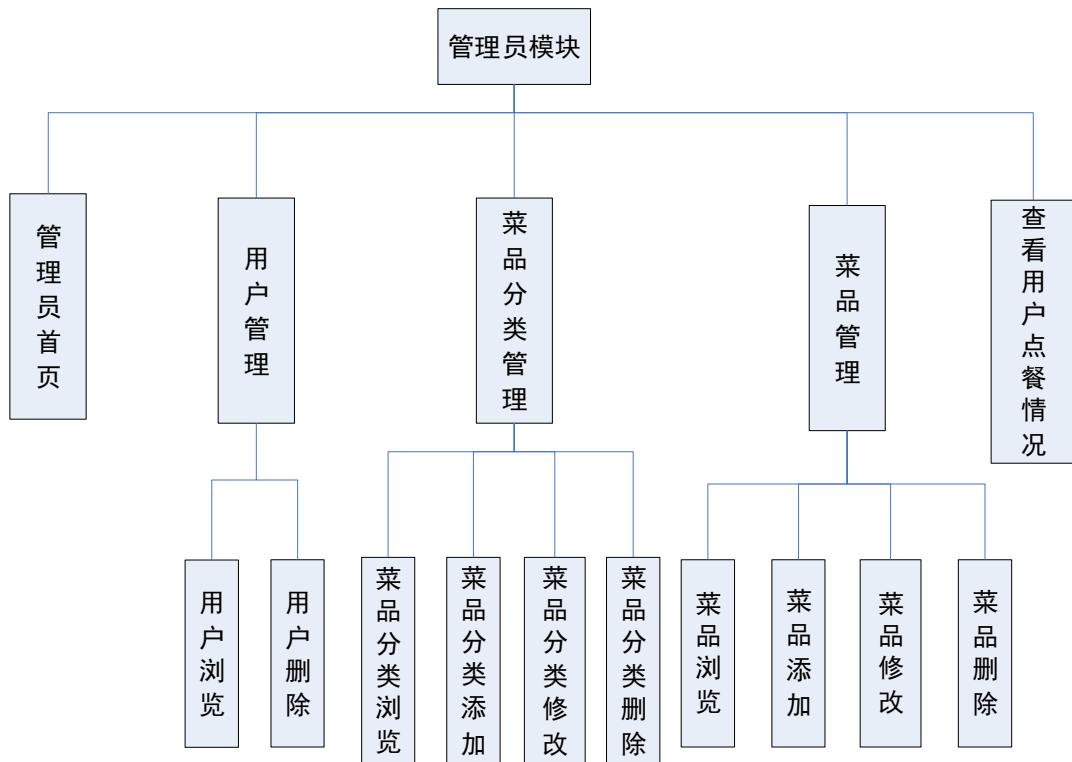


图4 管理员模块细化图

2、数据库设计：

从任务描述出发，结合模块设计，从中抽取关键要素设计 E-R 模型如图 5 所示。

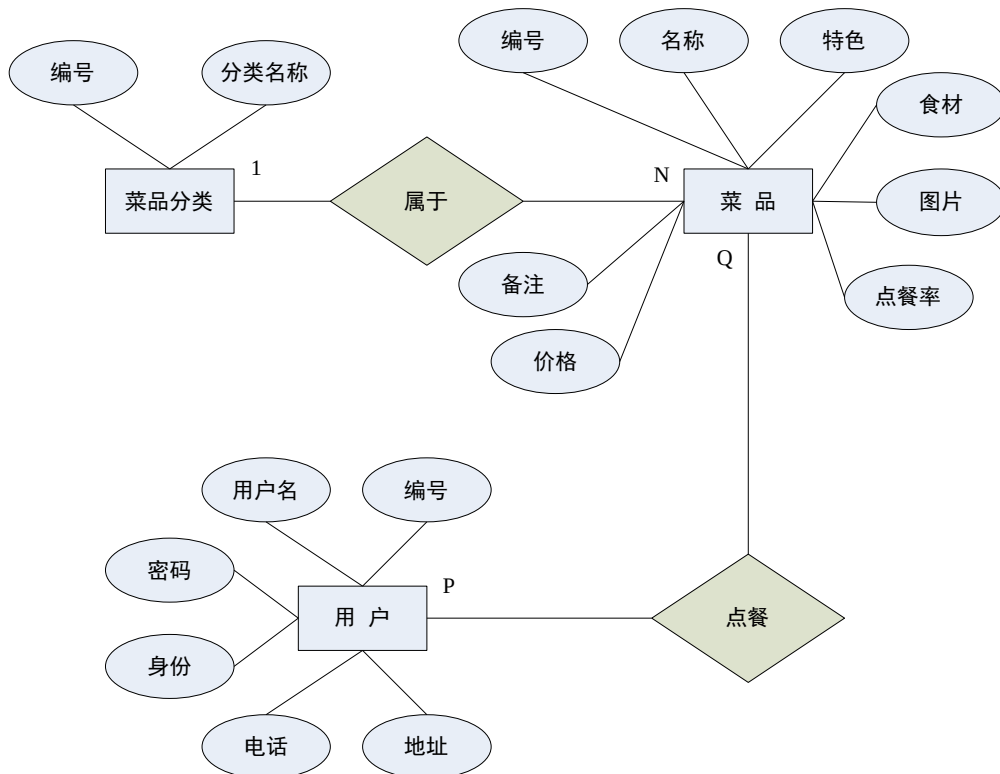


图5 网络点餐系统 E-R 图

对如上 E-R 图分析转换后，提取出 4 张数据库表，分别是：用户表、菜品分类表、菜品表和点餐表，表的具体设计见表 1-1 至 1-4。

表 1-1 user（用户表）

字段名	数据类型	主键	可为空	是否外键	其他约束	说明
id	int	是	否	否	无	用户编号
username	varchar(20)	否	否	否	唯一性	用户名
password	varchar(20)	否	否	否	无	密码
ident	char(1)	否	否	否	无	用户身份 (0：普通用户，1：管理员)
telephone	varchar(20)	否	否	否	无	电话
addresses	varchar(50)	否	否	否	无	地址

表 1-2 foodtype（菜品分类表）

字段名	数据类型	主键	可为空	是否外键	其他约束	说明
id	int	是	否	否	无	分类编号
typename	varchar(20)	否	否	否	唯一性	分类名称

表 1-3 food（菜品表）

字段名	数据类型	主键	可为空	是否外键	其他约束	含义
id	int	是	否	否	无	菜品编号
foodname	varchar(20)	否	否	否	唯一性	菜品名称
feature	varchar(100)	否	是	否	无	特色
material	varchar(100)	否	是	否	无	食材
price	int	否	否	否	无	价格
type	int	否	否	是	无	分类
picture	varchar(40)	否	是	否	无	图片
hits	int	否	否	否	无	点餐率
comment	int	否	否	否	无	备注（-1：正常菜，0：厨

字段名	数据类型	主键	可为空	是否外键	其他约束	含义
						师推荐菜品，正整数：特价菜品价格)

表 1-4 diningcar (点餐表)

字段名	数据类型	主键	可为空	是否外键	其他约束	含义
id	int	是	否	否	无	点餐编号
userid	int	否	否	是	无	用户编号
foodid	int	否	否	是	无	菜品编号

3、相关知识及过程与步骤：

任务 1：Java Web 开发和运行环境的搭建

(1) JDK 安装的具体步骤请参考“教学视频”中的“41.JDK 安装视频.mp4”。

(2) Tomcat 安装的具体步骤请参考“教学视频”中的“04. Tomcat 服务器简介、安装与配置(2-1).mp4”和“05.Tomcat 服务器简介、安装与配置(2-2).mp4”。

(3) Eclipse 安装和使用的具体步骤请参考“教学视频”中的“42.Eclipse 安装视频.mp4”和“43.Eclipse 使用视频.mp4”。

(4) MySQL 安装和配置的具体步骤请参考“教学视频”中的“44.MySQL 安装及配置视频.mp4”。

(5) Navicat 安装的具体步骤请参考“教学视频”中的“45.Navicat 安装视频.mp4”。

(6) 使用 Navicat 创建网络点餐系统的数据库 meal，具体步骤请参考“教学视频”中的“46.Navicat 使用视频.mp4”。SQL 脚本文件请参考资源中的脚本文件“meal 数据库脚本.sql”。

任务 2：网络点餐系统的页面原型

使用 HTML+CSS 实现网络点餐系统页面原型的设计，在实现页面功能的基础上，尽量美观。以下界面仅供参考，在保证页面内容的完整性的前提下，可以自由创新。

网络点餐系统的首页显示热点菜品、今日特价和厨师推荐菜品，每种菜品均提供超级链接以访问菜品详细信息，首页中还提供了登录和注册点餐系统的超

级链接，如图 6 所示。



图 6 网络点餐系统首页

在图 6 中点击菜品名称的超级链接将进入到菜品详细信息显示页面，如图 7 所示。

菜名	菠菜炒鸡蛋
特色	暂无
食材	主料：菠菜300克，鸡蛋3个，盐、料酒、葱末、姜末、味精、香油各适量。
类型	家常
价格	9元
图片	
点餐率	3次
备注	厨师推荐

将菜品添加到点餐车

图7 菜品详细信息页面

在图6中点击“登录”超级链接后，如图8所示。

用户登录

用户名	请输入用户名
密码	请输入密码
登录	

图8 登录页面

在图6中点击“注册”超级链接后，如图9所示。

用户注册

用户名:	
密码:	
电话:	
地址:	
注册	

图 9 注册页面

在图 8 中填写登录信息，点击“登录”按钮后，如果登录失败，则弹出如图 10 所示的提示窗口；登录成功后，如果是普通用户身份，将进入用户首页面，如图 11 所示；如果是管理员身份，将进入管理员首页面，如图 12 所示。

提示信息

×

用户名或密码错误!

确定

图 10 登录失败页面

网络点餐系统

正在点餐

我的点餐

user1

按菜名搜索

所有分类

搜索

#	菜名	特色	主料	价格	分类	图片	点餐率	备注	选择
1	菠菜炒鸡蛋	暂无	主料：菠菜300克，鸡蛋3个，盐、料酒、葱末、姜末、味精、香油各适量。	9	家常		3	厨师推荐	☐
2	韭菜炒鸡蛋	工艺：炒 口味：清香味	主料：韭菜4两（约160克），大鸡蛋3只。	8	家常		2		☐
3	小葱拌豆腐	特色：清香爽口，一清二白。	暂无	22	凉菜		2	特价18元	☐
4	渝味辣白菜	特色：色泽红亮、酸甜微辣，脆爽可口。	主料：娃娃菜200克。	6	家常		0		☐

将菜品添加到点餐车

<

1

2

3

4

5

6

7

8

9

10

11

>

图 11 用户首页面

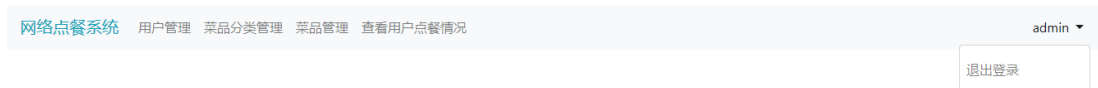


图 12 管理员首页面

在图 11 中选择菜品后，点击“将菜品添加到点餐车”按钮将进入如图 13 所示的结果页面，在结果页面中点击“确定”按钮将进入我的点餐页面，如图 14 所示。

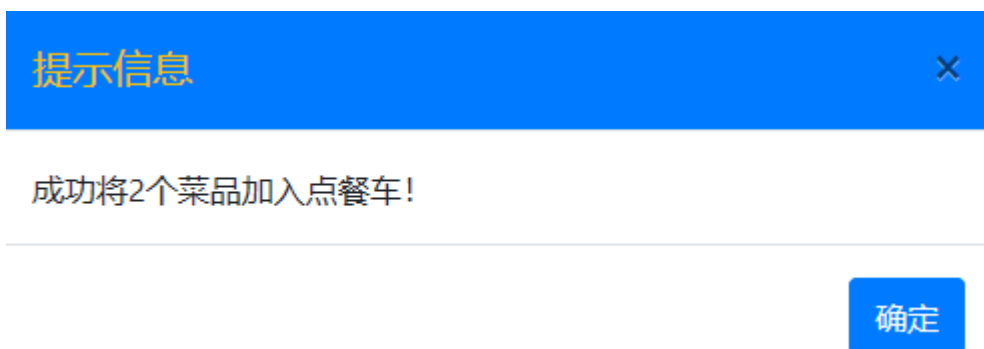


图 13 加入点餐车结果页面



图 14 我的点餐页面

在图 14 我的点餐页面，可以选择菜品从点餐车删除，这里不再截图赘述。

在图 11 的管理员首页面中点击导航条中的“菜品管理”超级链接将进入菜品管理页面，如图 15 所示。

网络点餐系统 用户管理 菜品分类管理 菜品管理 查看用户点餐情况 admin									
按菜名搜索		所有分类	搜索	添加菜品					
#	菜名	特色	主料	价格	分类	图片	点餐率	备注	操作
1	菠菜炒鸡蛋	暂无	主料：菠菜300克，鸡蛋3个，盐、料酒、葱末、姜末、味精、香油各适量。	9	家常		3	厨师推荐	修改 删除
2	韭菜炒鸡蛋	工艺：炒 口味：清香味	主料：韭菜4两（约160克），大鸡蛋3只。	8	家常		2		修改 删除
3	小葱拌豆腐	特色：清香爽口，一清二白。	暂无	22	凉菜		2	特价18元	修改 删除
4	渝味辣白菜	特色：色泽红亮、酸甜微辣，脆爽可口。	主料：娃娃菜200克。	6	家常		0		修改 删除
< 1 2 3 4 5 6 7 8 9 10 11 >									

图 15 菜品管理页面

在图 15 中，可以按菜名和分类查询菜品信息，还可以对菜品信息进行添加，修改和删除操作。在图 15 中，点击“添加菜品”按钮，页面如图 16 所示；点击“修改”按钮，页面如图 17 所示；点击“删除”按钮，弹出删除确认对话框，如图 18 所示。

添加菜品

菜品名称

菜品特色

主要原料

所属分类

主食

菜品价格

单位：元

菜品图片

浏览...

请选择上传的菜品图片，大小应小于5M，扩展名为jpg,png或gif。

菜品备注

-1

-1代表正常菜，0代表厨师推荐，正整数代表特价菜价格。

确认添加

放弃返回

图 16 添加菜品页面

修改菜品

菜品名称

菠菜炒鸡蛋

菜品特色

暂无

主要原料

主料：菠菜300克，鸡蛋3个，盐、料酒、葱末、姜末、蒜末、香油各适量。

所属分类

主食

菜品价格

9

单位：元

菜品图片

浏览...

请选择上传的菜品图片，大小应小于5M，扩展名为jpg.png或gif。



菜品备注

0

-1代表正常菜，0代表厨师推荐，正整数代表特价菜价格。

确认修改 放弃返回

图 17 修改菜品页面

删除确认

×

确定要删除该菜品吗？

放弃 删除

图 18 删除确认对话框

管理员的其他管理功能界面和菜品管理界面相似，这里不再截图赘述。

在图 12 的管理员首页面中点击导航条中的“查看用户点餐情况”超级链接将进入用户点餐列表页面，如图 19 所示。

user1	
小葱拌豆腐	¥22
韭菜炒鸡蛋	¥8
菠菜炒鸡蛋	¥9
合计	¥39

user2	
菠菜炒鸡蛋	¥9
韭菜炒鸡蛋	¥8
合计	¥17

图 19 用户点餐列表页面

下面以网络点餐系统的首页面为例，展示页面原型的制作过程，其他页面由学生自行实践完成。

第一步：创建 homepage.html 页面；

第二步：使用<h1> 标签展示页面 Logo，使用<a> 标签提供登录和注册链接；

第三步：<h1> 标签和<table><tr><td> 等标签完成“热点菜品”、“今日特价”和“厨师推荐”三个栏目的展示；

第四步：使用<a> 标签提供菜品详细信息链接；

第五步：调试、运行程序。检查显示结果是否正确。

任务 3：网络点餐系统的用户注册和登录

(1) 编写网络点餐系统的用户注册页面 register.html，界面可参考图 20。该页面提交给第 (2) 步中的 Servlet 处理。

用户注册



A registration form with four input fields and a submit button. The fields are labeled '用户名:' (Username), '密码:' (Password), '电话:' (Phone), and '地址:' (Address). The submit button is labeled '注册' (Register).

用户名:	
密码:	
电话:	
地址:	
注册	

图 20 注册界面

(2) 创建类名为 RegisterServlet 的 Servlet，其 mapping url 配置为：
/register。在该 Servlet 中，重写 doGet()方法，在其中读取用户输入的所有
信息并显示。

在读取信息之前，先进行编码转换：

```
request.setCharacterEncoding("utf-8");
```

读取单值参数如用户姓名时，使用如下语句：

```
String username=request.getParameter("un");
```

信息读取后进行输出，输出前，需要设置响应的内容类型：

```
response.setContentType("text/html;charset=utf-8");
```

获取当前的输出流对象时，使用如下语句：

```
PrintWriter out=response.getWriter();
```

向页面输出信息时，使用如下语句：

```
out.println(username);
```

(3) 编写网络点餐系统的用户登录页面 login.html，界面可参考图 21。该页
面提交给第 (4) 步中的 Servlet 处理。

欢迎登录



A login form with two input fields and a submit button. The first field is labeled '用户名' (Username) and has a placeholder '请输入用户名' (Please enter username). The second field is labeled '密码' (Password). The submit button is labeled '登录' (Login).

用户名	请输入用户名
密码	
登录	

图 21 登录页面

(5) 创建类名为 LoginServlet 的 Servlet，其 mapping url 配置为：
/login。在该 Servlet 中，重写 doGet() 方法，读取用户的输入信息。判断用户名和密码是否正确（假定用户名为 neusoft，密码为 123），如果正确，则跳转到页面 user_index.html 中，如果错误，则显示“用户名或密码错误”。
跳转可以使用如下语句：

```
response.sendRedirect("user_index.html");
```

(6) user_index.html 页面可使用任务 2 的代码。

(7) 调试、运行程序。检查显示结果是否正确。

任务 4：网络点餐系统的菜品分类管理

(1) 创建 Java 类 util.DBUtil.java，封装连接数据库的操作。参考代码如下：

```
package util;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class DBUtil {
    // 数据库驱动程序
    private String driver;
    // 数据库连接信息
    private String url;
    // 连接数据库的用户名
    private String username;
    // 连接数据库的密码
    private String password;
    // 连接对象
    private Connection con;
    // 预编译语句对象
    private PreparedStatement pstmt;
    // 查询结果分页时，每页显示记录数。
    public static final long PAGE_REC_NUM = 8;
```

```
public void setDriver(String driver) {
    this.driver = driver;
}

public void setUrl(String url) {
    this.url = url;
}

public void setUsername(String username) {
    this.username = username;
}

public void setPassword(String password) {
    this.password = password;
}

public DBUtil() {
    driver = "com.mysql.cj.jdbc.Driver";
    url = "jdbc:mysql://localhost:3306/meal?characterEncoding=utf8&serverTimezone=UTC&useSSL=false"; // 数据库名meal 可变
    username = "root";
    password = "root"; // 数据库密码可变
}

// 初始化方法，加载驱动程序，获得数据库的连接对象。
private void init() {
    try {
        Class.forName(driver);
        con = DriverManager.getConnection(url, username, password);
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// 数据库操作对象的关闭
private void close() {
    if (pstmt != null) {
        try {
            pstmt.close();
        }
    }
}
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    if (con != null) {
        try {
            con.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

// 为预编译的 SQL 语句中的占位符?设置值，值被放在字符串数组 params 中。
private void setParams(String[] params) {
    if (params != null) {
        for (int i = 0; i < params.length; i++) {
            try {
                pstmt.setString(i + 1, params[i]);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

// 执行更新类（增删改）的 SQL 语句，SQL 语句中含有占位符。
public int update(String sql, String[] params) {
    int result = 0;
    init();
    try {
        pstmt = con.prepareStatement(sql);
        setParams(params);
        result = pstmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return result;
}
```

```
// 执行更新类（增删改）的 SQL 语句，SQL 语句中不含有占位符。
public int update(String sql) {
    return update(sql, null);
}

// 执行返回多条记录的查询操作，结果被封装到 List 中。
public List<Map<String, String>> getList(String sql, String[] params) {
    List<Map<String, String>> list = null;
    init();
    try {
        pstmt = con.prepareStatement(sql);
        setParams(params);
        ResultSet rs = pstmt.executeQuery();
        list = getListFromRS(rs);
        rs.close();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return list;
}

public List<Map<String, String>> getList(String sql) {
    return getList(sql, null);
}

// 执行返回至多一条记录的查询操作，结果被封装到 Map 中。
public Map<String, String> getMap(String sql, String[] params) {
    Map<String, String> m = null;
    List<Map<String, String>> l = getList(sql, params);
    if (l != null && l.size() != 0) {
        m = (Map<String, String>) (l.get(0));
    }
    return m;
}

public Map<String, String> getMap(String sql) {
    return getMap(sql, null);
}

// 将结果集中的内容封装到 List 中
private List<Map<String, String>> getListFromRS(ResultSet rs) throws
SQLException {
```

```
List<Map<String, String>> list = new ArrayList<Map<String, String>>();
ResultSetMetaData rsmd = rs.getMetaData();
while (rs.next()) {
    Map<String, String> m = new HashMap<String, String>();
    for (int i = 1; i <= rsmd.getColumnCount(); i++) {
        String colName = rsmd.getColumnLabel(i);
        String s = rs.getString(colName);
        if (s != null) {
            m.put(colName, s);
        }
    }
    list.add(m);
}
return list;
}
```

// 查询结果分页时，返回分页信息的 Map，包括总页数，每页记录数和当前页中的记录。

```
public Map<String, Object> getPage(String sql, String[] params, String curPage) {
    Map<String, Object> page = new HashMap<String, Object>();
    String newSql = sql + " limit " + (Long.parseLong(curPage) - 1) * PAGE_REC_NUM + "," + PAGE_REC_NUM;
    List<Map<String, String>> pageList = getList(newSql, params);
    sql = sql.toLowerCase();
    String countSql = "";
    if (sql.indexOf("group") >= 0) {
        countSql = "select count(*) as tempNum from (" + sql + ") as temp";
    } else {
        countSql = "select count(*) as tempNum " + sql.substring(sql.indexOf("from"));
    }
    String count_s = (String) getMap(countSql, params).get("tempNum");
    long count = Long.parseLong(count_s);
    long totalPage = 0;
    if (count % PAGE_REC_NUM == 0)
        totalPage = count / PAGE_REC_NUM;
    else
        totalPage = count / PAGE_REC_NUM + 1;
    page.put("list", pageList);
    page.put("totalPage", totalPage);
}
```

```

        page.put("recNum", PAGE_REC_NUM);
        return page;
    }

    public Map<String, Object> getPage(String sql, String curPage) {
        return getPage(sql, null, curPage);
    }
}

```

(2) 创建 Java 类 service.FoodTypeService.java，封装菜品分类相关操作的业务逻辑，提供菜品分类添加，删除，修改，根据 id 读取分类信息，读取全部或部分分类信息的方法，该类使用 (1) 中的类完成数据库操作。

(3) 创建 Servlet 程序 FoodTypeList.java，使用 (2) 中的类读取所有菜品分类信息并显示，同时提供按照分类名称的模糊查询，提供添加，修改和删除分类信息的链接。

(4) 创建 type_add.html 页面，提供添加分类信息的 form 表单。

(5) 创建 Servlet 程序 FoodTypeAdd.java，使用 (2) 中的类将分类信息插入到数据库中，并提示操作结果及下一步操作的提示。

(6) 创建 Servlet 程序 FoodTypeEdit.java，使用 (2) 中的类读取待修改的菜品分类信息，显示在 form 表单中，供用户修改。

(7) 创建 Servlet 程序 FoodTypeUpdate.java，使用 (2) 中的类实现分类信息的修改，并提示操作结果及下一步操作的提示。

(8) 创建 Servlet 程序 FoodTypeDel.java，使用 (2) 中的类实现分类信息的删除，并提示操作结果及下一步操作的提示。

(9) 调试、运行程序。检查显示结果是否正确。

任务 5：网络点餐系统的菜品分类管理的优化

(1) 创建 result.jsp 页面，使用 EL 读取并显示 Servlet 传递的提示信息，链接地址和链接上的文字。

```

<%@ page pageEncoding="utf-8" %>
<!DOCTYPE html>
<html>
<head>
<title>操作结果页</title>
</head>
<body>

```

```
<h3>${requestScope.msg }</h3>
<a href="${requestScope.addr }">${requestScope.text }</a>
</body>
</html>
```

(2) 修改任务 4 中的 Servlet 程序 FoodTypeList.java, 读取所有菜品分类信息并存储在请求对象中, 同时将请求转发给 type_list.jsp 页面。

```
request.setCharacterEncoding("utf-8");
String s_tn = request.getParameter("s_tn");
// 调用模型获取数据
List<Map<String,String>> types = new
FoodTypeService().getTypeList(s_tn);
request.setAttribute("types", types);
request.getRequestDispatcher("type_list.jsp").forward(request,
response);
```

(3) 创建 type_list.jsp 页面, 使用<c:forEach> 标签显示 (2) 中类传递的分类信息, 并提供按照分类名称的模糊查询, 提供添加, 修改和删除分类信息的链接。

[illegible]

```

        <th>序号</th>
        <th>分类名称</th>
        <th>操作</th>
    </tr>
    <c:forEach var="type" items="${types }"
varStatus="vs">
        <tr>
            <td>${vs.count }</td>
            <td>${type.typename }</td>
            <td><a href="type_edit?id=${type.id }">修改
</a> <a
                        onclick="return confirm('确定要删除该分类
吗? ')" href="type_del?id=${type.id }">删除</a></td>
        </tr>
    </c:forEach>
</table>
</div>
</c:when>
<c:otherwise>暂无菜品分类! </c:otherwise>
</c:choose>
</body>
</html>

```

(4) 修改任务 4 中的 Servlet 程序 FoodTypeAdd.java，将分类信息插入到数据库中，并将提示信息，链接地址和链接文字存储在请求对象中，同时将请求转发给 result.jsp 页面。

```

request.setCharacterEncoding("utf-8");
// 读取菜品分类名称
String typename = request.getParameter("typename");
// 调用模型
int r = new FoodTypeService().addType(typename);
if(r == 1 ) {
    request.setAttribute("msg", "添加菜品分类成功!");
}else {
    request.setAttribute("msg", "添加菜品分类失败!");
}
request.setAttribute("addr", request.getContextPath() +
"/type_list");
request.setAttribute("text", "查看分类列表");
// 请求转发的通用的结果页面
request.getRequestDispatcher("/result.jsp").forward(request,
response);

```


(5) 修改任务 4 中的 Servlet 程序 FoodTypeEdit.java，读取待修改的菜品分类信息并存储在请求对象中，同时将请求转发给 type_edit.jsp 页面。

```
request.setCharacterEncoding("utf-8");
// 读取待修改的分类id
String id = request.getParameter("id");
Map<String,String> type = new FoodTypeService().getTypeById(id);
// 将待展示的信息存储到request中
request.setAttribute("type", type);
// 显示到修改表单中（交给JSP展示）
request.getRequestDispatcher("type_edit.jsp").forward(request, response);
```

(6) 创建 type_edit.jsp 页面，使用 EL 表达式显示待修改的分类信息，并供用户输入修改。

```
<%@ page pageEncoding="utf-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>修改菜品分类</title>
</head>
<body>
<c:if test="${not empty requestScope.type }">
    <div align="center">
        <h1>修改菜品分类</h1>
        <hr>
        <form action="type_update" method="post">
            分类名称: <input type="text" name="typename" value="$
{type.typename }" required>
            <input type="hidden" name="id" value="${type.id }">
            <input type="submit" value="修改">
        </form>
    </div>
</c:if>
<c:if test="${empty type }">
    该分类不存在!
</c:if>
</body>
</html>
```

(7) 修改任务 4 中 Servlet 程序 FoodTypeUpdate.java, 实现分类信息的修改, 并将提示信息, 链接地址和链接文字存储在请求对象中, 同时将请求转发给 result.jsp 页面。可参考 (4) 中的代码完成。

(8) 修改任务 4 中 Servlet 程序 FoodTypeDel.java, 实现分类信息的删除, 并将提示信息, 链接地址和链接文字存储在请求对象中, 同时将请求转发给 result.jsp 页面。可参考 (4) 中的代码完成。

(9) 调试、运行程序。检查显示结果是否正确。

(五) 成果提交

学生提交完成的项目代码及运行效果截图到平台。

(六) 考核方式与标准

1、考核方法：

- (1) 学生通过平台进行成果提交。
- (2) 学生助教检查项目成果并给出相应分数。

2、考核标准：

本项目占形成性考核成绩中的 10 分。

优秀 (9-10 分)：在规定时间内正确安装和配置 Web 应用服务器, 数据库服务器及集成开发环境。正确实现网络点餐系统的全部静态页面, 准确使用 Servlet 完成对用户注册信息的读取和显示, 完成用户登录的判断和跳转, 使用 JDBC 完成菜品分类的管理, 并正确使用 JSTL 对分类管理页面实施优化。

良好 (7-8 分)：在规定时间内正确安装和配置 Web 应用服务器, 数据库服务器及集成开发环境。较好的实现网络点餐系统的全部静态页面, 登录注册功能及分类管理功能, 在实现中存在瑕疵。

合格 (6 分)：安装和配置 Web 应用服务器, 数据库服务器及集成开发环境时存在个别错误。基本实现网络点餐系统的静态页面, 登录注册功能, 分类管理功能, 且无严重错误。

不合格 (0-5 分)：未在规定时间内正确安装和配置 Web 应用服务器, 数据库服务器及集成开发环境。未提交三级项目或在实现网络点餐系统的静态页面, 登录注册, 分类管理时有严重错误。